Part - 1

1. How does object-oriented programming improve software maintainability compared to procedural programming ?

Ans:

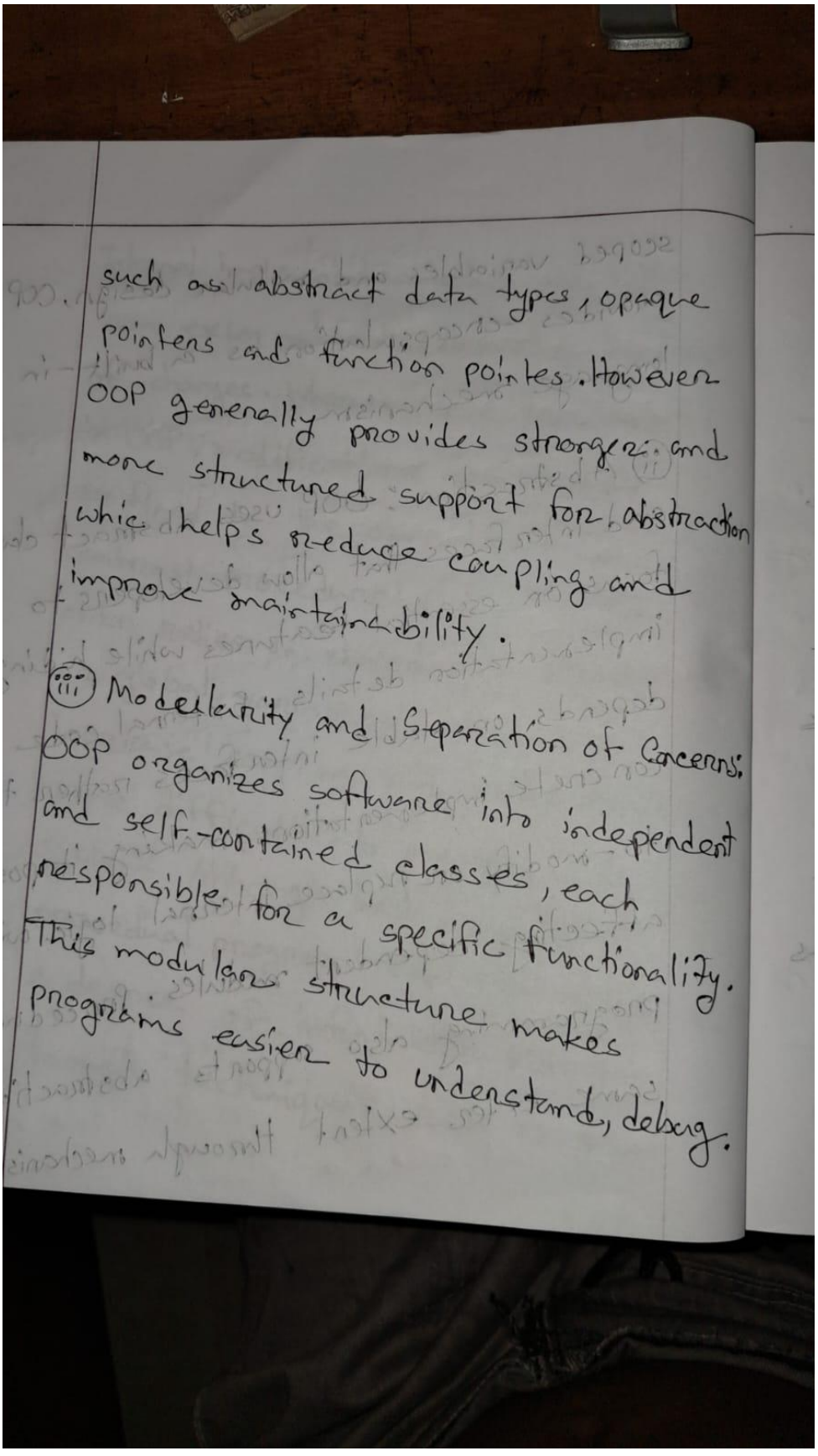
Object-oriented programming improve software maintainability compared to procedural programming through several key mechanism that enhance code organization, reusability, flexibility and ease of modification.

i) Encapsulation: OOP combines data and the methods that operate on that data into a single unit called a class.

Internal implementation details are hidden from external code, reducing the impact of changes. When internal logic changes in OOP, modifications are usually confined to the class itself. In contrast, procedural programming often relies on shared data structures or global variables, which can increase coupling and make changes more difficult to manage in large system. Although well-designed procedual programs can also achieve a degree of information hiding through techniques such as file-

scoped variables and modular design, OOP provides encapsulation as a built-in language mechanism.

(ii) Abstraction: OOP uses abstract classes and interfaces that allow developers to focus on essential features while hiding implementation details. External code depends on stable interfaces rather than concrete implementations, making it possible to modify or replace internal logic without affecting dependent modules. Procedural programming also supports abstraction to some extent through mechanisms



such as abstract data types, opaque pointers and function pointes. However OOP generally provides stronger and more structured support for abstraction which helps reduce coupling and improve maintainability.

(iii) Modularity and Separation of Concerns:
OOP organizes software into independent and self-contained classes, each responsible for a specific functionality. This modular structure makes programs easier to understand, debug.

It also enables multiple developers to work on different components simultaneously with minimal interference. In procedural programming, functionality is often distributed across multiple functions and modules. which can become difficult to manage as the system grows in size and complexity.

(iv) Inheritance and Polymorphism: OOP supports inheritance and polymorphism making it easier to extend existing system while minimizing modifications to existing code. These features promote code

reuse and reduce duplication. Through interfaces, abstract classes, and polymorphism, developers can more easily follow the Open/Closed Principle by extending functionality without extensively modifying existing code. However, inheritance alone does not guarantee adherence to the Open/Closed Principle and if misused can introduce maintenance problems such as the fragile base class problem. Compared to procedural programming where new functionality

often requires modifying existing functions and adding conditional logic.

(v) Localized Side Effects: Data and behavior are encapsulated within objects, changes to an objects state are localized, making software easier to understand, debug and test. Procedural programming often relies on shared mutable data or global variable, which can make it difficult to trace how and where data is modified. The localization of side effects in OOP reduces the likelihood of unintended

interactions and simplifies maintenance.

(vi) Easier Refactoring and Unit Testing: The modular structure and clear boundaries of OOP make refactoring and unit testing more manageable. Individual classes can be tested independently injection. This improves software reliability and gives developers greater confidence when modifying existing code. In procedual programming, higher coupling and shared state can make testing and refactoring more difficult and error-prone.

2. A class contains one instance variable and one static variable. If 100 objects are created from that class, how many copies of each variable will exist in memory? Explain your answer.

Ans:

If a class contains one instance variable and one static variable and 100 objects are created from that class:

(i) Instance variable: 100 copies will exist in memory.

(ii) Static variable: 1 copy will exist in memory.

Explain:

An instance variable belongs to each individual object. Therefore every object

created from the class gets its own separate copy of the instance variable. So, for 100 objects, there will be 100 copies of the instance variable.

A static variable belongs to the class itself, not to individual objects. All objects share the same static variable. Therefore, regardless of how many object are created, there will be only one copy of the static variable in memory.

3. Why is a constructor not considered a regular method in Java? Discuss at least three differences between constructors and method.

Ans

In Java, a construction is not considered a regular method because its fundamental purpose, invocation mechanism and JVM-level treatment are fundamentally different. While both are written in class bodies, a constructor is a special initialization block, whereas a method defines the behavioral capabilities of an object.

At the bytecode level, constructor are

compiled into a special JVM method named <init>, which has unique rules that standard method do not follow.

Here are three critical difference between constructors and regular method

Construction	Method
① Used to initialize an object when it is created.	Used to perform specific operations or behaviors
② Called automatically when an object is created using the new keyword.	Called explicitly by the programmer
③ Cannot be inherited by subclasses.	Can be inherited and overridden

4. A student class has three overloaded constructors:

Student ()

Student (String name)

Student (String name, int id)

Explain how Java decides which constructor to invoke when an object is created.

Ans:

In Java the decision of which overloaded constructor to invoke is made at compile time, not at runtime. This process is called constructor overloading resolution or static binding. When an object is created using the new keyword, the Java compiler selects the appropriate constructor by

examining the number of arguments, their types and their order.

① Matching the Number of arguments:
The compiler first checks the number of arguments passed to the constructor.

Student S1 = new Student();
This invokes:
Student()
because no arguments are provided.

Student S2 = Student("Alice");
The invokes:
Student(String name)
because one String argument is provided.

Student S3 = new Student("Alice", 101);
This invokes:

Student (String name, int id)
because two arguments are provided: a String followed by an int.

(ii) Matching the Types of Arguments: It multiple constructors have the same number of parameters the compiler check whether the argument types match the parameter types and selects the most specific match.

(iii) Conversion Rules: It no exact match exists, Java attempts conversions in the following order —

① Exact match
② Widening primitive conversion

3. Auto boxing

4. Variable argument

(iv) Special case : Null

The value null can be assigned to any reference type. Therefore when null is passed Java chooses the most specific matching reference-type constructor.

5. Compare local variable, instance variable and class variable in terms of:

(i) Scope

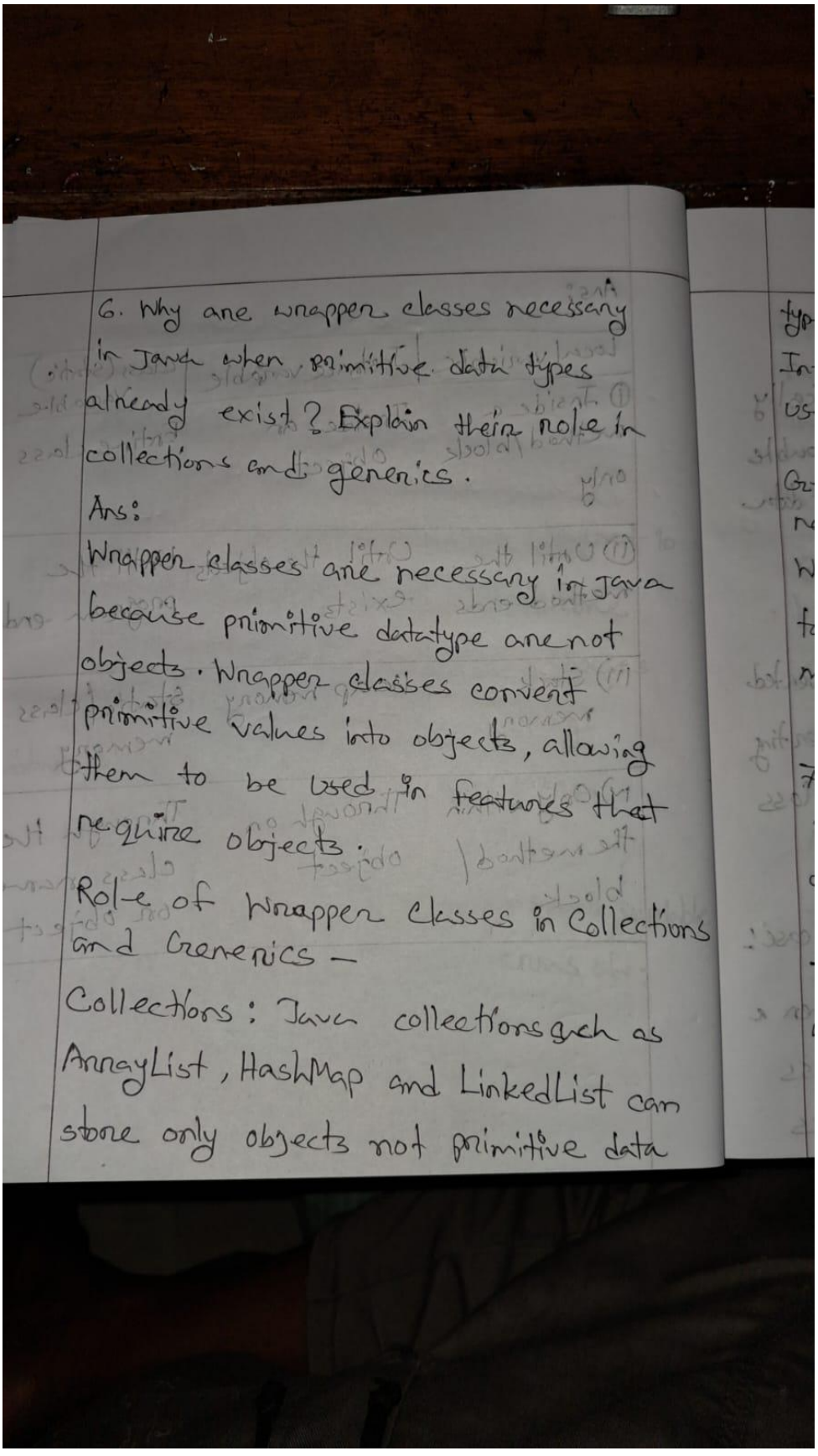
(ii) Lifetime

(iii) Memory allocation

(iv) Accessibility

Ans:

Local variable	Instance variable	Class (Static) variable
(i) Inside a method/block only	Inside an object	Entire class
(ii) Until the method ends	Until the objects exists	Until the program ends
(iii) Stack memory	Heap memory	Static/class memory
(iv) Only within the method/block	Through an object	Through the class name or object



6. Why are wrapper classes necessary in Java when primitive data types already exist? Explain their role in collections and generics.

Ans:

Wrapper classes are necessary in Java because primitive datatype are not objects. Wrapper classes convert primitive values into objects, allowing them to be used in features that require objects.

Role of Wrapper classes in Collections and Generics –

Collections: Java collections such as ArrayList, HashMap and LinkedList can store only objects not primitive data

types. Therefore wrapper classes like Integer, Double and Character are used.

Generics: Generics work only with reference type, not primitive types. Wrapper classes allow primitive values to be used with generic classes and method.

7. Consider the following code:

```
int a = 10
double b = a
int c = (int) b;
```

Identify which conversions are implicit and which are explicit. Why does Java require explicit casting in some cases?

Ans: double b = a;
Implicit Conversion: int is automatically converted to double because double can store all int value without data loss.

int c = (int) b;
Explicit Conversion: double is converted to int using a cast because converting from double to int may cause loss of data.

Java require explicit casting in case:
Java require explicit casting when a conversion may result in data loss or loss of precision. This ensures

that the programmer is aware of the potential risk and intentionally performs the conversion.

8. What is the difference between a class and an object? Explain why multiple objects created from the same class can have different states but share the same behavior.

Ans:

Class	Object
i) blueprint or template	instance of a class
ii) It defines variable and method	It stores actual data and uses method
iii) Example - Student	Example - S1, S2

Object have different status but the same behavior.

Different states: Each object has its own copy of instance variable. So they can store different value.

Same behavior: All object share the same methods defined the class.

9. Autoboxing and Unboxing make programming easier, but they can also introduce performance overhead. Explain how autoboxing/unboxing works and discuss situations where excessive use may affect program performance.

Ans:

Autoboxing and unboxing are features in Java that automatically convert between primitive data type and their corresponding wrapper classes.

Autoboxing = Automatic conversion of a primitive type to its wrapper object.

Example- int → Integer

Unboxing = Automatic conversion of a wrapper object to its primitive type.

Example- Integer → int

Example=

```
int a = 10
Integer b = a // autoboxing
int c = b; // unboxing
```

Performance Overhead:

Excessive use of autoboxing and unboxing can affect performance because -

① Creating wrapper objects requires additional memory allocation.

② Automatic conversions increase CPU overhead.

Situations Performance may be Affected

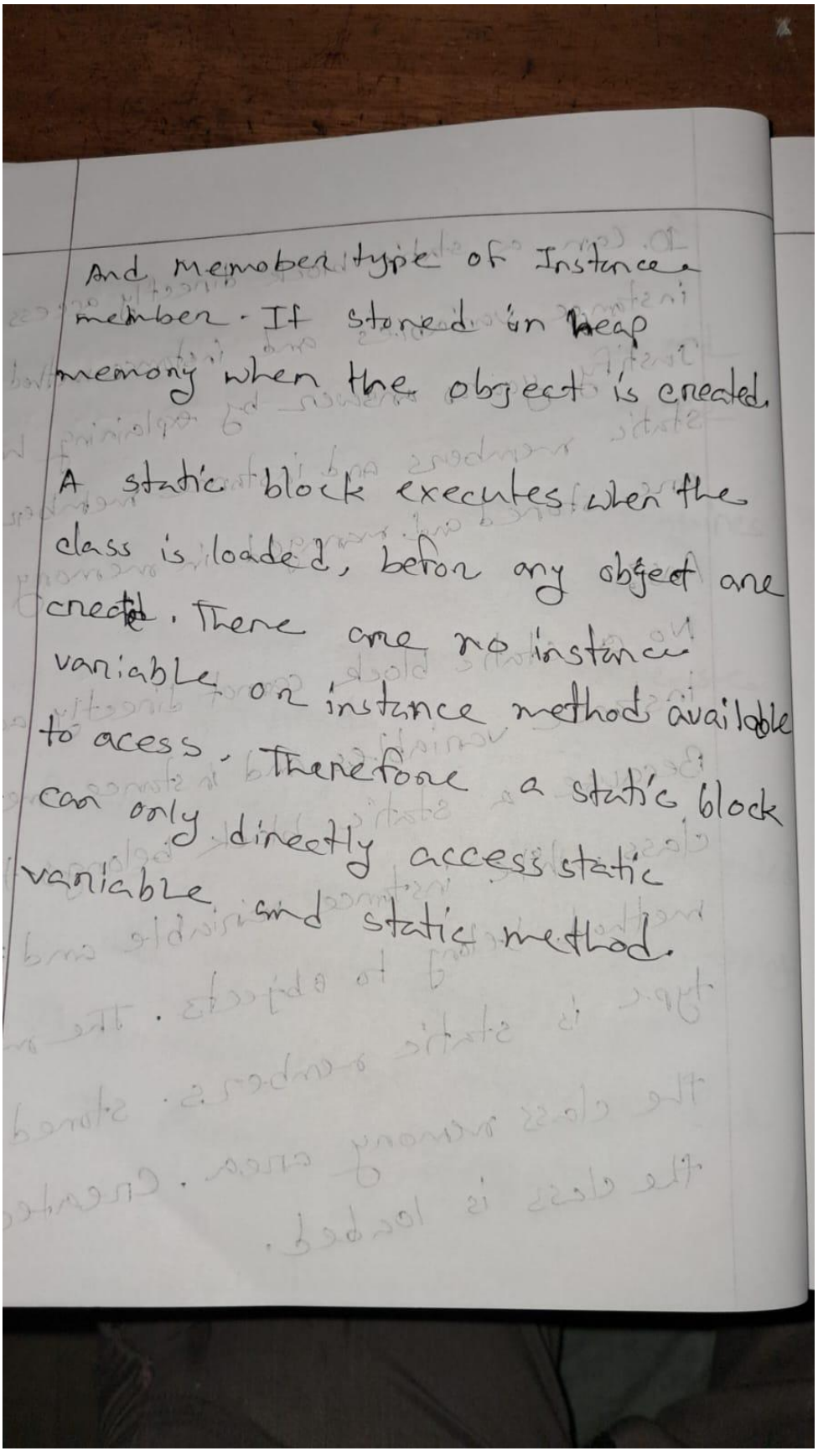
① Large loops with repeated boxing and unboxing operation

② Processing large collections such as ArrayList<Integer>.

10. Can a static block directly access instance variables and instance method? Justify yours answer by explaining how static members and instance members are stored and managed in memory.

Ans:

No a static block cannot directly access instance variables and instance method. Because a static block belongs to the class, while instance variable and instance method belong to objects. The memory type is static members. stored in the class memory area. Created when the class is loaded.



And member type of Instance member. It stored in heap memory when the object is created.

A static block executes when the class is loaded, before any object are created. There are no instance variable, or instance method available to acess. Therefore a static block can only directly access static variable and static method.